Laboratory Guide 3.6: Shor's Algorithm

Quantum Factoring via Period Finding and the QFT

Course: Quantum Computing Foundation

Contents

1 Introduction and Objectives **2**

2 Part A: The Classical Foundation **2**
2.1 RSA and the Factoring Problem 2
2.2 Reduction: Factoring ⇒ Order Finding 2

3 Part B: The Quantum Fourier Transform **3**
3.1 Definition 3
3.2 Circuit Structure 4
3.3 The QFT and Periodicity 4

4 Part C: The Quantum Period-Finding Subroutine **5**
4.1 Circuit Overview 5
4.2 Full State-Vector Evolution ($N = 15$, $a = 7$, $n = 4$) 5
4.3 Continued-Fractions Extraction 6

5 Part D: Building the QFT (IBM Quantum Composer) **6**
5.1 Exercise 1: Build and Verify a 2-Qubit QFT 6
5.2 Exercise 2: Period Detection (Conceptual Composer Exercise) 7

6 Part E: Coding Shor's Algorithm (Qiskit) **7**
6.1 Exercise 3: Python Implementation 7

7 Part F: Assessment (Quiz) **9**

1 Introduction and Objectives

In Lab 3 and Lab 3.5 we learned to exploit quantum interference for global function evaluation (Deutsch–Jozsa) and unstructured search (Grover). In this laboratory we tackle the problem that gave quantum computing much of its public profile: **integer factorisation**. Shor's Algorithm (1994) factors an n-bit integer N in time $\mathcal{O}(n^3)$, while the best known classical algorithms require $\mathcal{O}\left(e^{1.9\,n^{1/3}}\right)$ (the General Number Field Sieve). This exponential separation underpins the security of every RSA-encrypted message on the internet today.

Prerequisites:

- Lab 3.5: *Grover's Algorithm and Amplitude Amplification*
- Lesson QC14: *Quantum Algorithms (Grover & Shor)*
- Basic modular arithmetic (congruences, gcd)

Learning Objectives:

1. Understand the classical reduction from factoring to *order finding*.
2. Master the Quantum Fourier Transform (QFT) — its matrix, circuit, and role.
3. Trace the full state-vector evolution of the period-finding subroutine.
4. Implement a complete Shor simulation in Qiskit for $N = 15$, $a = 7$.
5. Interpret measured bitstrings via the continued-fractions algorithm.

Important: The Exponential Speedup

The Exponential Speedup — What Is Really at Stake

RSA-2048 (used for most HTTPS traffic today) relies on the fact that factoring a 2048-bit number is computationally infeasible classically. The GNFS would take $\sim 10^{23}$ operations on a classical supercomputer: longer than the age of the universe. Shor's Algorithm would factor the same number in $\sim (2048)^3 \approx 8.6 \times 10^9$ quantum operations — a few hours on a fault-tolerant quantum computer. This is not a polynomial speedup; it is an *exponential* one in the exponent of the classical bound.

2 Part A: The Classical Foundation

2.1 RSA and the Factoring Problem

RSA encryption is built on a simple asymmetry: **multiplying** two large primes p and q to produce $N = p \cdot q$ is trivially fast, but **factoring** N back into p and q is (classically) exponentially hard. The security of the system relies entirely on this asymmetry.

2.2 Reduction: Factoring ⇒ Order Finding

The key mathematical insight behind Shor's Algorithm is that the hard problem of factoring can be *reduced* to the (classically hard but *quantum-easy*) problem of finding the **multiplicative order** of an integer.

Multiplicative Order

Let $N \in \mathbb{Z}^+$ and $a \in \mathbb{Z}$ with $\gcd(a, N) = 1$. The **multiplicative order** (or **period**) of a modulo N is the smallest positive integer r such that

$$a^r \equiv 1 \pmod{N}.$$

Equivalently, r is the period of the function $f(x) = a^x \bmod N$.

The full classical reduction proceeds in three steps:

1. **Choose** a random a with $2 \leq a \leq N - 1$. Check $g = \gcd(a, N)$. If $g > 1$, we already found a factor — done.
2. **Find the order** r of a modulo N (this is the quantum step).
3. **Extract the factors:** If r is even and $a^{r/2} \not\equiv -1 \pmod{N}$, then by Euler's theorem and the factorisation of $x^2 - 1 = (x-1)(x+1)$:

$$p = \gcd\left(a^{r/2} - 1,\ N\right), \qquad q = \gcd\left(a^{r/2} + 1,\ N\right)$$

are non-trivial factors of N.

Classical Reduction: N

1. $\gcd(7, 15) = 1$ — no trivial factor; proceed to step 2.
2. Compute $f(x) = 7^x \bmod 15$:

x	0	1	2	3	4	5	6	7	8
$f(x)$	1	7	4	13	1	7	4	13	1

The sequence repeats with period $r = 4$.

3. $r = 4$ is even; check $7^{4/2} = 7^2 = 49 \equiv 4 \not\equiv -1 \pmod{15}$. Condition satisfied.

$$p = \gcd(7^2 - 1,\ 15) = \gcd(48,\ 15) = 3$$

$$q = \gcd(7^2 + 1,\ 15) = \gcd(50,\ 15) = 5$$

Verification: $3 \times 5 = 15$ ✓

3 Part B: The Quantum Fourier Transform

3.1 Definition

The **Quantum Fourier Transform** (QFT) is the quantum analogue of the Discrete Fourier Transform (DFT). Acting on an n-qubit basis state $|x\rangle$:

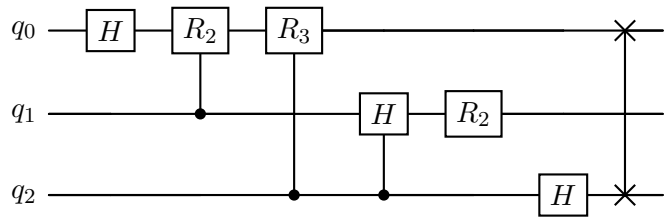
$$\mathrm{QFT}_N |x\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \omega^{xk} |k\rangle, \qquad \omega = e^{2\pi i/N}, \quad N = 2^n. \tag{1}$$

It maps each basis state to a uniform superposition with phases that encode the frequency content of x. Critically, it is *unitary* and can be implemented with only $\mathcal{O}(n^2)$ gates (vs. $\mathcal{O}(N \log N)$ for the classical FFT).

3.2 Circuit Structure

The QFT on n qubits decomposes into a cascade of Hadamard gates and controlled phase-rotation gates R_k, where $R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix}$, followed by a bit-reversal via SWAP gates.

The circuit for $n = 3$ qubits is shown below:



3.3 The QFT and Periodicity

The key property we exploit is: *the QFT maps a periodic state to concentrated peaks in frequency space.*

Periodic State

A state of the form $|\psi_r\rangle = \frac{1}{\sqrt{K}} \sum_{j=0}^{K-1} |jr + s\rangle$ (with period r, offset s, and $K = \lfloor N/r \rfloor$) has amplitudes concentrated at multiples of N/r after applying QFT_N.

QFT of a Periodic State (r

Take $|\psi_0\rangle = \frac{1}{2}\big(|0\rangle + |4\rangle + |8\rangle + |12\rangle\big)$. Applying QFT_{16}:

$$\mathrm{QFT}_{16}\,|\psi_0\rangle = \frac{1}{2} \cdot \frac{1}{4} \sum_{k=0}^{15} \left[\sum_{j=0}^{3} e^{2\pi i(4j)k/16} \right] |k\rangle = \frac{1}{8} \sum_{k=0}^{15} \left[\sum_{j=0}^{3} e^{\pi i j k/2} \right] |k\rangle \,.$$

The inner sum is a geometric series:

$$\sum_{j=0}^{3} e^{\pi i j k/2} = \begin{cases} 4 & \text{if } k \equiv 0 \pmod 4 \\ 0 & \text{otherwise.} \end{cases}$$

Therefore, $\mathrm{QFT}_{16}\,|\psi_0\rangle = \frac{1}{2}\big(|0\rangle + |4\rangle + |8\rangle + |12\rangle\big)$, with measurement probabilities concentrated at $\{0, 4, 8, 12\} = \{k \cdot N/r \mid k = 0, 1, 2, 3\}$. **The period $r = 4$ is encoded in the spacing of the peaks.**

4 Part C: The Quantum Period-Finding Subroutine

4.1 Circuit Overview

The algorithm uses two quantum registers:

- **Counting register** (n qubits): stores the superposition of exponents $x \in \{0, \ldots, 2^n - 1\}$. We need $2^n \geq N^2$; for $N = 15$ we use $n = 4$ ($2^4 = 16$).
- **Work register** (m qubits, $m = \lceil \log_2 N \rceil$): stores $f(x) = a^x \bmod N$. For $N = 15$, $m = 4$ qubits.

The circuit has three stages:

$$\underbrace{H^{\otimes n}}_{\text{(1) Superposition}} \longrightarrow \underbrace{U_f}_{\text{(2) Modular Exp.}} \longrightarrow \underbrace{\mathrm{QFT}^{-1}}_{\text{(3) Frequency Extraction}} \longrightarrow \underbrace{\text{Measure}}_{\text{(4) Readout}}$$

4.2 Full State-Vector Evolution ($N = 15$, $a = 7$, $n = 4$)

Complete State Evolution

Step 1 — Superposition. Apply $H^{\otimes 4}$ to the counting register, work register initialised to $|1\rangle$:

$$|\psi_1\rangle = \frac{1}{4} \sum_{x=0}^{15} |x\rangle_C \otimes |1\rangle_W .$$

Step 2 — Oracle (Modular Exponentiation). The unitary $U_f |x\rangle_C |y\rangle_W = |x\rangle_C |y \cdot a^x \bmod N\rangle_W$ entangles the registers:

$$|\psi_2\rangle = \frac{1}{4} \sum_{x=0}^{15} |x\rangle_C \otimes \underbrace{|7^x \bmod 15\rangle_W}_{\text{cycles: 1,7,4,13,1,7,...}} .$$

The work register cycles through $\{1, 7, 4, 13\}$ with period $r = 4$.

Step 3 — Measure work register (conceptual). Suppose the outcome is $|1\rangle_W$ (this occurs for $x \in \{0, 4, 8, 12\}$). The counting register collapses to the uniform periodic superposition:

$$|\psi_3\rangle = \frac{1}{2}\big(|0\rangle + |4\rangle + |8\rangle + |12\rangle\big)_C \otimes |1\rangle_W .$$

Note: in the actual circuit the work register is traced out implicitly by the QFT; explicit measurement is not required.

Step 4 — Inverse QFT. As computed in Part B, the IQFT concentrates probability at $\{0, 4, 8, 12\} = \{k \cdot 16/4 \mid k = 0, 1, 2, 3\}$:

$$\mathrm{QFT}_{16}^{-1} |\psi_3\rangle = \frac{1}{2}\big(|0\rangle + |4\rangle + |8\rangle + |12\rangle\big)_C .$$

Before the QFT, the counting register *marginal* was uniform (1/16 for each state). The QFT concentrates the probability into $r = 4$ peaks — this is the quantum advantage.

Step 5 — Measurement and Classical Post-processing. One of $\{0, 4, 8, 12\}$ is measured with equal probability 1/4. The measured value y encodes the phase $\phi = y/2^n$.

4.3 Continued-Fractions Extraction

Continued-Fractions Algorithm

Given measured value y, compute the best rational approximation $p/q \approx y/2^n$ with denominator $q \leq N$. The denominator q is the period estimate $\hat{r}$.

Example outcomes:

y	$y/16$	Best approx	$\hat{r}$	$7^{\hat{r}} \bmod 15$	Result
0	0	0/1	1	7	Retry (trivial)
4	1/4	1/4	4	1	✓
8	1/2	1/2	2	4	Retry, try $2r$
12	3/4	3/4	4	1	✓

Success probability per run: $\sim$75%. Expected trials to success: $\sim$1.3.

5 Part D: Building the QFT (IBM Quantum Composer)

Platform: IBM Quantum Composer — `https://quantum.ibm.com/composer`

5.1 Exercise 1: Build and Verify a 2-Qubit QFT

The 2-qubit QFT maps: $|00\rangle \mapsto \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$, $|01\rangle \mapsto \frac{1}{2}(|00\rangle + i\,|01\rangle - |10\rangle - i\,|11\rangle)$, etc.

Student Activity

Circuit Instructions:

1. Start with both qubits in $|0\rangle$.
2. Apply H to q_0 (MSB).
3. Apply S ($= R_2 = P(\pi/2)$) to q_0, controlled on q_1.
4. Apply H to q_1.
5. Apply `SWAP` between q_0 and q_1 (bit-reversal).
6. **Add a measurement** and observe the statevector on the Q-Sphere.

Expected: All four states $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ with equal probability 25%. Now **change the initial state to** $|01\rangle$ (add an X gate on q_1 before step 1) and verify the phases rotate as predicted.

5.2 Exercise 2: Period Detection (Conceptual Composer Exercise)

Student Activity

Observe the Oracle Effect:

1. Put 4 qubits in uniform superposition (H on all).
2. Inspect the Q-Sphere — all 16 states equally bright.
3. Now apply a "period-4 phase oracle": apply Z to q_0 (MSB) only. This modifies the phase of states with $q_0 = 1$, creating a partial phase structure.
4. Apply the H gate to q_0 again and observe how the probability on $q_0 = 0$ doubles (constructive interference) while $q_0 = 1$ vanishes — a 1-qubit Fourier transform!

Insight: The full Shor oracle followed by QFT does exactly this across all n qubits simultaneously, but for the *period-r subspace*.

6 Part E: Coding Shor's Algorithm (Qiskit)

6.1 Exercise 3: Python Implementation

The full implementation searches for the period of $f(x) = 7^x \bmod 15$ using $n = 4$ counting qubits and a 2-qubit orbit-encoded work register.

Coding Task: QFT Circuit

```python
import math
from qiskit import QuantumCircuit

def build_qft(n: int) -> QuantumCircuit:
    """
    Constructs an n-qubit Quantum Fourier Transform circuit.
    Uses Hadamard + controlled-phase (CP) gates + SWAP.
    """
    qc = QuantumCircuit(n)
    for i in range(n):
        qc.h(i)
        for j in range(i + 1, n):
            angle = math.pi / (2 ** (j - i))
            qc.cp(angle, j, i)   # controlled phase R_{j-i+1}
    # Bit-reversal: swap qubits
    for i in range(n // 2):
        qc.swap(i, n - 1 - i)
    return qc
```

Coding Task: Full Order-Finding Circuit for N

```python
from qiskit import QuantumRegister, ClassicalRegister
import fractions

def build_shor_circuit() -> QuantumCircuit:
    """
    Order-finding for a=7, N=15 using:
    - 4 counting qubits (register C)
    - 2 work qubits encoding the orbit {1,7,4,13} as {00,01,10,11}

    Modular exponentiation via controlled cyclic shifts:
      C[0] controls U^1 (add 1 mod 4): CCNOT(C0,W0,W1) + CNOT(C0,W0)
      C[1] controls U^2 (add 2 mod 4): CNOT(C1,W1)
      C[2] controls U^4 = I: nothing
      C[3] controls U^8 = I: nothing
    """
    qr_c = QuantumRegister(4, 'count')
    qr_w = QuantumRegister(2, 'work')
    cr   = ClassicalRegister(4, 'c')
    qc   = QuantumCircuit(qr_c, qr_w, cr)

    # 1. Superposition on counting register
    qc.h(qr_c)
    qc.barrier()

    # 2. Controlled modular exponentiation
    #    c-U^1: controlled by count[0]
    qc.ccx(qr_c[0], qr_w[0], qr_w[1])
    qc.cx(qr_c[0],  qr_w[0])
    #    c-U^2: controlled by count[1]
    qc.cx(qr_c[1],  qr_w[1])
    #    c-U^4 = I (count[2]) and c-U^8 = I (count[3]): omitted
    qc.barrier()

    # 3. Inverse QFT on counting register
    qft_inv = build_qft(4).inverse()
    qc.compose(qft_inv, qubits=qr_c, inplace=True)
    qc.barrier()

    # 4. Measure counting register
    qc.measure(qr_c, cr)
    return qc
```

Coding Task: Run and Extract the Period

```python
from qiskit_aer import AerSimulator

def extract_period(y: int, n_count: int, N: int = 15) -> int:
    """
    Use the continued-fractions algorithm to find the period r
    from a measured bitstring y (integer) and n_count counting qubits.
    """
    phase = y / (2 ** n_count)
    frac  = fractions.Fraction(phase).limit_denominator(N)
    return frac.denominator   # period estimate

# Run the simulation
qc  = build_shor_circuit()
sim = AerSimulator()
job = sim.run(qc, shots=2000)
counts = job.result().get_counts()

print("Measurement Results:")
for state, shots in sorted(counts.items(), key=lambda x: -x[1]):
    y   = int(state, 2)
    r   = extract_period(y, 4)
    pct = shots / 2000 * 100
    print(f"  |{state}> (y={y:2d})  ->  r_hat = {r}  ({pct:.1f}%)")

# Factor extraction
r = 4
a = 7
p = math.gcd(a ** (r // 2) - 1, 15)
q = math.gcd(a ** (r // 2) + 1, 15)
print(f"\nFactors of 15: gcd(7^2-1, 15) = {p}, gcd(7^2+1, 15) = {q}")
```

Expected output: Peaks at 0000, 0100, 1000, 1100 (values $0, 4, 8, 12$), each $\approx 25\%$. Extracted period $r = 4$. Factors: $\{3, 5\}$.

7 Part F: Assessment (Quiz)

Q1. The Speedup

What is the time complexity of Shor's Algorithm for factoring an n-bit integer?

A) $\mathcal{O}(2^n)$

B) $\mathcal{O}(n^3)$

C) $\mathcal{O}(\sqrt{n})$

Q2. Role of the QFT

Why is the inverse QFT applied to the counting register at the end of the algorithm?

A) To disentangle the counting and work registers before measurement.

B) To convert the uniform probability distribution of the counting register into peaks at multiples of $2^n/r$, from which the period r can be efficiently extracted.

C) To apply amplitude amplification as in Grover's Algorithm.

Q3. Failure Mode

Shor's Algorithm is run for $N = 15$, $a = 7$, $n = 4$. A measurement yields $y = 0$. What should the algorithm do?

A) Report $r = 0$ and halt.

B) Report $r = N = 15$.

C) Discard the result and re-run; $y = 0$ is an uninformative outcome.

Q4. Continued Fractions

A measurement returns $y = 8$ from $n = 4$ counting qubits ($2^n = 16$). What period $\hat{r}$ does the continued-fractions algorithm return, and how should it be interpreted?

A) $\hat{r} = 2$; since $7^2 \not\equiv 1 \pmod{15}$, try $2\hat{r} = 4$.

B) $\hat{r} = 4$; directly compute factors.

C) $\hat{r} = 8$; the measurement is a higher harmonic.

Solutions: Q1: B ($\mathcal{O}(n^3)$, polynomial in the number of bits of N); Q2: B (the QFT converts periodic structure into measurable frequency peaks); Q3: C ($y = 0$ gives $\phi = 0$ whose CF expansion yields $r = 1$, trivially true for all a — no information gained, repeat); Q4: A ($8/16 = 1/2 \Rightarrow \hat{r} = 2$; verify $7^2 \bmod 15 = 4 \neq 1$, so try $r = 2\hat{r} = 4$; confirm $7^4 \bmod 15 = 1$).